

Graphs, Networks, and Flow Problems as Linear Programs

1 Graphs and Networks

Graphs and **Networks** are useful tools to model many real-world structures and problems. How would you explain a geographical map to, say, the GPS of your car? The first thing you will tell it is that there are special points on the map that represent cities or intersections of roads. We are going to call those points **vertices** or **nodes**. Second, you will tell it that some nodes (possibly, all) are connected by roads. These connections we will call **edges**. Third, you specify how far two cities connected by an edge are. This could be the physical distance, the time needed to travel between them, etc., depending on the parameters you are interested in. We will call those numbers **weights**. A graph is a formalization of those concepts: it is a collection of vertices and edges, and it is denoted by $G = G(V, E)$, with V being the set of vertices and E the set of edges. To distinguish it from another graph that has a different set of nodes and edges, we may denote the latter by $G' = G(V', E')$. The letter G can be replaced by others; for instance, we will often denote trees (a special class of graphs, to be defined in a later class) by $T = T(V, E)$. The set of vertices of a generic graph G is usually denoted by $V(G)$, its cardinality by $|V|$ or n . The set of edges of a generic graph G is usually denoted by $E(G)$, its cardinality by $|E|$ or m . An edge $e \in E$ is denoted by the two vertices it connects. So, if it connects u and v , we write $e = uv$. We also say that u and v are the **endpoints** of e . When we consider a graph with weights w on the edges, we will say that $G(V, E)$ is an **edge-weighted** graph. Sometimes, it will be interesting to consider graphs with weights on nodes. In this case, we will talk about **node-weighted** graphs. The prefixes edge- and node- will be omitted if it is clear from the context to which of the two cases we refer. When a graph G has an edge between any pair of nodes, we say that it is **complete**. The **degree** of a node is the number of edges incident to it. In our “GPS example”, this would translate to the number of roads that go out of a city.

A (directed) **network** $N(V, A)$ is a graph where edges have a direction, and they are called **arcs**, and their set is denoted by A . The arc (u, v) goes from u to v . We say that u is the **tail** and v the **head** of the arc. As mentioned above, edges/arcs may have **weight**: a weight of arc $a = (u, v)$ is a real number denoted $c(a)$ or $c(u, v)$, c_a , c_{uv} . In the example on the left shown below,

- vertices $V = \{1, 2, 3, 4, 5, 6\}$ and arcs $A = \{(1, 2), (1, 3), (2, 5), (4, 2), (5, 3), (5, 6)\}$
- We have **weights** on the arcs: $c(1, 2) = 2$, $c(1, 3) = 5$, $c(2, 5) = 1$, $c(4, 2) = 3$, $c(4, 6) = 2$, $c(5, 3) = 1$, $c(5, 6) = 2$.

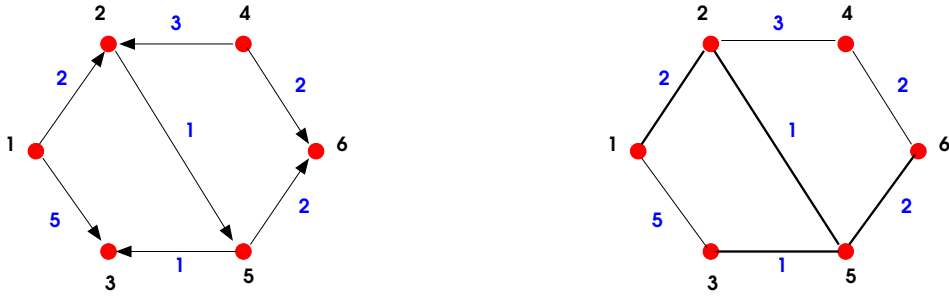
On the right we have the graph obtained from the network on the left by removing the direction from its arcs.

A sequence of distinct nodes and edges/arcs from the graph/network $v_1, e_1, v_2, e_2, \dots, v_{k-1}, e_k, v_k$ is

- a **path** (directed path) if each e_i goes from v_i to v_{i+1} .
- a **chain** (undirected path) if each e_i goes v_i and v_{i+1} or from v_{i+1} to v_i .

(often we write: $e_1, e_2, \dots, e_k$ is a path (we omit vertices) or write: $v_1, v_2, \dots, v_k$ is a path (we omit edges/arcs))

A network is **connected** if for every two nodes there is a path connecting them; otherwise it is **disconnected**. A **cycle** is an undirected path starting and ending in the same node, never repeating any other node or edge. A **directed cycle** is a direct path starting and ending in the same node, never repeating any other node or edge.



2 Flow problems

A key concept in many network problems is that of *flows*, representing (nonnegative) units of a single good that are being transferred between nodes of a network. In flow problems, the goal is to transfer those units respecting the constraints imposed by the network while at the same time optimizing an objective function, that often models either the *cost* of transferring the good, or the *amount* of goods being transferred. Let us now introduce some specific examples of flow problems.

2.1 Minimum Cost Flow

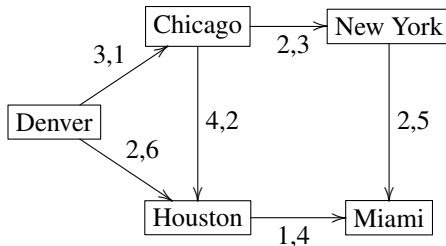
In the Minimum Cost Flow problem we are given:

- A network $N(V, A)$;
- For each node $v \in V$ a demand (if positive) / offer (if negative) d_v ;
- For each arc $a \in A$, a per-unit cost c_a and a capacity u_a ,

and we aim to find

- A flow satisfying all demand / requests of the nodes and respecting capacities, at minimum cost.

Consider the instance given by the network $N(V, A)$ below,



where, on every edge, the first number denotes the capacity and the second denotes the cost. We moreover have the following demands/offers.

City v	Denver	Chicago	Houston	New York	Miami
d_v	-4	-2	3	1	2

One readily checks that a feasible solution is given by the following vector of flow (we abbreviate the edge from Denver to Chicago as DC, and similarly for the others):

$$x_{DC} = 3, \quad x_{DH} = 1, \quad x_{CH} = 3, \quad x_{CN} = 2, \quad x_{HM} = 1, \quad x_{NM} = 1.$$

Indeed, nonnegative units of flows are transferred on every arc; moreover, capacities are respected, and the amount of flow that enters any v minus the amount of flow exiting it coincides with d_v , for every node v of the graph. The cost of this solutions is given by:

$$3 \cdot 1 + 1 \cdot 6 + 3 \cdot 2 + 2 \cdot 3 + 1 \cdot 4 + 1 \cdot 5 = 30.$$

More generally, we can find an optimal solution for the Minimum Cost Flow problem above as a solution to the Linear Program below

$$\begin{aligned} \min \quad & c_{DC} \cdot x_{DC} + c_{DH} \cdot x_{DH} + c_{CH} \cdot x_{CH} + c_{CN} \cdot x_{CN} + c_{HM} \cdot x_{HM} + c_{NM} \cdot x_{NM} \\ & \underbrace{\sum_{a=(u,v) \in A} x_a}_{\text{flow into } v} - \underbrace{\sum_{a=(v,u) \in A} x_a}_{\text{flow out of } v} = d_v \quad \text{for } v \in \{D, C, H, N, M\} \\ & x_a \leq u_a \quad \text{for all } a \in A \\ & x \geq 0 \end{aligned}$$

The equalities are usually called the *flow balance* or *flow conservation* constraints. The first set of inequalities are called *capacity constraints* while the second set are called *nonnegativity*.

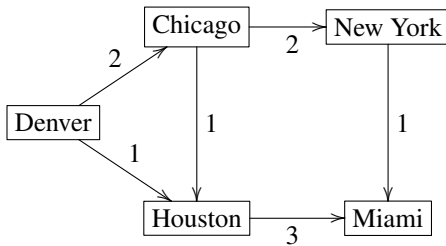
More generally, an LP formulation for a generic Minimum Cost Flow problem is the following.

$$\begin{aligned} \min \quad & \sum_{a \in A} c_a x_a \\ & \underbrace{\sum_{a=(u,v) \in A} x_a}_{\text{flow into } v} - \underbrace{\sum_{a=(v,u) \in A} x_a}_{\text{flow out of } v} = d_v \quad \text{for all } v \in V \\ & x_a \leq u_a \quad \text{for all } a \in A \\ & x \geq 0 \end{aligned}$$

Note that a necessary condition for the feasibility of a Minimum Cost Flow is that $\sum_{v \in V} d_v = 0$. This is, however, not sufficient: the capacities may not be large enough to allow us to satisfy the demands / offers.

2.2 Maximum Flow

Consider the following setting: A delivery company runs a delivery network between major US cities. Selected cities are connected by routes as shown below. On each route a number of delivery trucks is dispatched daily (indicated by labels on the corresponding edges). A customer is interested in hiring the company to deliver his products daily from Denver to Miami, and needs to know how much product can be delivered on a daily basis.



This problem can be formulated as a linear program as follows:

$$\begin{aligned}
& \text{maximize} && x_{DC} + x_{DH} \\
& x_{DC} && -x_{CH} - x_{CN} && = 0 \\
& x_{DH} && +x_{CH} && -x_{HM} && = 0 \\
& && x_{CN} - x_{NM} && = 0 \\
& 0 \leq x_{DC} \leq 2 \\
& 0 \leq x_{DH} \leq 1 \\
& 0 \leq x_{CH} \leq 1 \\
& 0 \leq x_{CN} \leq 2 \\
& 0 \leq x_{NM} \leq 1 \\
& 0 \leq x_{HM} \leq 3
\end{aligned}$$

The equations are called *flow balance / conservation* constraints, and imply that, in any node other than Denver and Miami, the amount of flow entering the node must equal the amount of flow leaving the node. The inequalities are *nonnegativity* and *capacity constraints*.

In general, we will be given:

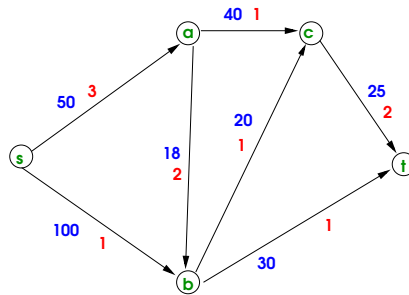
- a network $N = (V, A)$;
- a *source* s (Denver in the example above) and a *sink* t (Miami in the example above);
- a *capacity* u_a for every arc $a \in A$,

and we want to find a flow given by the optimal solution to the linear program below:

$$\begin{aligned}
& \max \quad \underbrace{\sum_{a=(s,u) \in A} x_a}_{\text{flow out of } s} - \underbrace{\sum_{a=(u,s) \in A} x_a}_{\text{flow into } s} \\
& \underbrace{\sum_{a=(u,v) \in A} x_a}_{\text{flow into } v} - \underbrace{\sum_{a=(v,u) \in A} x_a}_{\text{flow out of } v} = 0 \quad \text{for all } v \in V \setminus \{s, t\} \\
& 0 \leq x_a \leq u_a \quad \text{for all } a \in A.
\end{aligned}$$

2.3 Maximum Flow over time

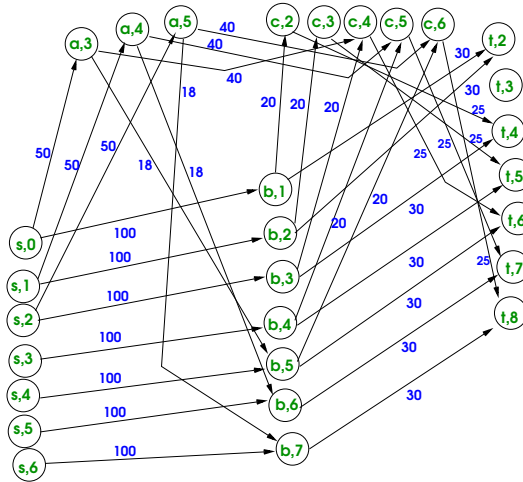
Consider the flow problem depicted in the following figure.



We want to send a maximum amount of flow from s to t ; the arc capacities are shown in blue. If that were all the information we have, we would conclude that the maximum flow is 55 units by solving the corresponding LP given in the previous section. However, the numbers shown in red are *time delays*, given in a consistent time unit, say hours.

For example, it takes one hour for flow leaving node a to reach node c . The kind of question we can ask is; how much flow can we send from s to t in the span of, say, 8 hours? To fix ideas, we will assume that flow must depart any node at the start of an hour. What we will show next is how to reduce this kind of problem to a standard max flow problem. It turns out that we can think of a couple of different versions of this problem.

Version 1. In this version, flow cannot be held up at a node, that is to say, as soon as flow arrives at a node it must immediately depart. Consider the following figure.



In this figure we have created multiple versions of each node. For each node i of the original network, now we have multiple copies labeled by a pair (i, h) where h indicates the *hour* at which flow arrives at node i . We don't need all such nodes. For example we don't need nodes (t, h) for $h > 8$ since the time horizon is eight hours. And we don't need nodes (t, h) for $h < 2$ since the earliest we can ever get to t is at hour 2. Similarly, we don't have nodes (a, h) for $h < 2$ since the earliest we can ever get to a is at $h = 3$. Likewise, we don't have (a, h) for $h > 5$ since it takes a minimum of 3 hours to get from a to t .

Also note how the arcs are arranged. For example, we have arcs of going from (a, h) to $(b, h + 2)$, so as to account for the time it takes to travel from a to b . A final detail is that all the arcs from a node (i, h) to a node (j, h') have the same capacity, which is equal to the original capacity between i and j (these are the blue numbers). It is clear that a max flow from the set of s nodes to the set of t nodes is what we need to compute.

Version 2. In this version, flow cannot be delayed at any node if we so desire – by any number of hours. This could be an advantage: flow waits until capacity becomes available. To handle this version, we make a simple adaptation to the above time-expanded network. Namely, for any node (i, h) we add an arc to $(i, h + 1)$, with infinite capacity.

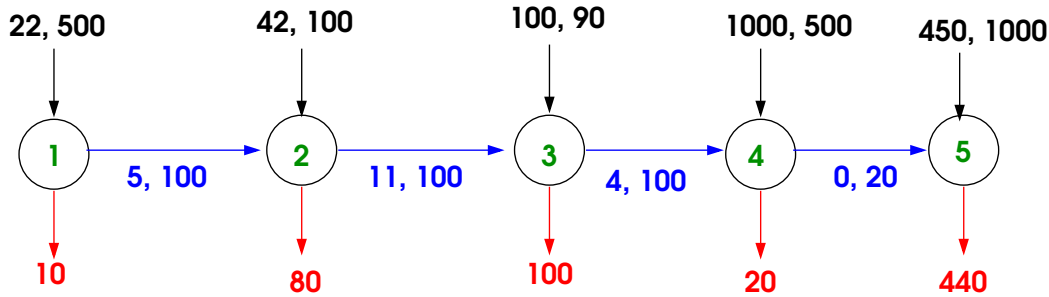
For example, in the case of the three “ a ” nodes, we get the following structure:



Here the purple arcs have infinite capacity. Notice that this allows one to have flow arrive at node a at time 3, and then wait *two* hours at node a before departing at time 5. Likewise, we will have a chain of purple arcs connecting the seven “ b ” nodes in the network under Version 1.

2.4 A logistic problem

The picture below shows a 6-period problem with demands in red. The blue quantities are per-unit inventory costs (on the left) and capacities (on the right), and per-unit production costs (on the left) and capacities (on the right) are shown in black. The goal is to satisfy all demands respecting capacity and production constraints, at minimum cost.



To formulate the problem as a linear program, let us introduce the following variables:

- x_t = production at time t , for $t = 1, \dots, 5$;
- y_t inventory kept between time t and $t + 1$, for $t = 1, \dots, 4$;

We can then write the following linear program that solves our problem

$$\begin{array}{ll}
 \min & 22x_1 + 42x_2 + 100x_3 + 1000x_4 + 450x_5 + 5y_1 + 11y_2 + 4y_3 \\
 & x_1 - y_1 = 10 \\
 & x_2 + y_1 - y_2 = 80 \\
 & x_3 + y_2 - y_3 = 100 \\
 & x_4 + y_3 - y_4 = 20 \\
 & x_5 + y_4 = 440 \\
 & x_1 \leq 500 \\
 & x_2 \leq 100 \\
 & x_3 \leq 90 \\
 & x_4 \leq 500 \\
 & x_5 \leq 1000 \\
 & y_1 \leq 100 \\
 & y_2 \leq 100 \\
 & y_3 \leq 100 \\
 & y_4 \leq 20 \\
 & x, y \geq 0
 \end{array}$$

In the LP above, the objective function minimizes the costs. The equality constraints imply that the units of the good available at a certain time minus the units that are stored in the inventory at that time equals the demand. We then have upper bounds on production and inventory, and nonnegativity constraints. Note that this can be interpreted as a minimum cost flow problem, where